

LAPORAN PRAKTIKUM
STRUKTUR DATA
Single Linked List

disusun Oleh:

MUHAMAD FAJRI AULIA
NIM 2511532009

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
PADANG, 5 MEI 2026

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kelima dengan judul “**Single Linked List**”. Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai Single Linked List dengan java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 7 Mei 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM	1
KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR.....	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Persyaratan Praktikum.....	1
1.4 Waktu dan tempat pelaksanaan	2
BAB II PEMBAHASAN.....	3
2.1 Pengertian Stack	3
2.2 Langkah Pengerjaan	3
2.2.1 NodeSLL.....	3
2.2.2 TambahSLL	4
2.2.3 Pencarian SLL	7
2.2.4 Hapus SLL	8
BAB III KESIMPULAN DAN SARAN	12
3.1 Kesimpulan.....	12
3.2 Saran	12
BAB IV LAPORAN TUGAS.....	13
4.1 Kode Program	13
4.2 Driver Program.....	13
4.3 Output yang dihasilkan	17
DAFTAR PUSTAKA	19

DAFTAR GAMBAR

Gambar 1.....	6
Gambar 2.....	8
Gambar 3.....	11
Gambar 4.....	17
Gambar 5.....	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Single Linked List sesuai dengan namanya merupakan salah satu jenis dari struktur data Linked List. Linked List sendiri sering disebut sebagai senarai berantai. Masing- masing data dalam LinkedList disebut sebagai node yang menempati alokasi memori secara dinamis. Sementara itu, Single Linked List adalah sebuah linked list yang menggunakan sebuah variabel pointer saja untuk menyimpan banyak data. Pada implementasinya single linked list ini dapat diimplementasikan dalam 2 variasi yaitu circular dan nocircular.

1.2 Tujuan Praktikum

- Memahami konsep dasar dari struktur data Single Linked List.
- Mengidentifikasi penggunaan Single Linked List dalam program.
- Mengimplementasikan metode bawaan dari Single Linked List.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut :

- Telah mengikuti perkuliahan teori Struktur data sebagai dasar pemahaman.
- Membawa perlengkapan yang diperlukan, antara lain laptop atau computer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Enviroment (IDE) yang direkomendasikan.
- Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
- Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
- Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan tempat pelaksanaan

Pelaksanaan praktikum Java mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing masing, selama memenuhi syarat teknis yang ditetapkan.

BAB II

PEMBAHASAN

2.1 Pengertian Stack

Single Linked List sesuai dengan namanya merupakan salah satu jenis dari struktur data Linked List. Linked List sendiri sering disebut sebagai senarai berantai. Masing- masing data dalam LinkedList disebut sebagai node yang menempati alokasi memori secara dinamis. Sementara itu, Single Linked List adalah sebuah linked list yang menggunakan sebuah variabel pointer saja untuk menyimpan banyak data. Pada implementasinya single linked list ini dapat diimplementasikan dalam 2 variasi yaitu circular dan nocircular.

2.2 Langkah Pengerjaan

Untuk lebih memahami kita akan mulai membahas program berikut.

2.2.1 NodeSLL

Untuk kode programnya sebagai berikut.

```
package pekan5_2511532009;

public class NodeSLL_2511532009 {
    // node bagian data
    int data_2009;
    // pointer ke node berikutnya
    NodeSLL_2511532009 next_2009;
    // konstruktor menginisialisasi node dengan data
    public NodeSLL_2511532009 (int data_2009) {
        this.data_2009 = data_2009;
        this.next_2009 = null;
    }
}
```

Pada program ini kita mendeklarasikan sebuah node bagian data kemudian kita yang mana kemudian kita dapat melihat pada program bahwa kondisi awal dari next adalah null atau kosong. Selain itu disini kita menggunakan konstruktor yang menginisialisasi node dengan data.

2.2.2 TambahSLL

Untuk program dari Tambah SLL driver ada sebagai berikut.

```
package pekan5_2511532009;

public class TambahSLL_2511532009 {
    public static NodeSLL_2511532009 insertAtFront_2009(NodeSLL_2511532009
head_2009, int value_2009) {
        NodeSLL_2511532009 new_node_2009 = new NodeSLL_2511532009(value_2009);
        new_node_2009.next_2009 = head_2009;
        return new_node_2009;
    }

    // fungsi menambahkan node di akhir SLL
    public static NodeSLL_2511532009 insertAtEnd_2009(NodeSLL_2511532009
head_2009, int value_2009) {
        //buat sebuah node dengan sebuah nilai
        NodeSLL_2511532009 newNode_2009 = new NodeSLL_2511532009(value_2009);
        // jika list kosong maka node jadi head
        if (head_2009 == null) {
            return newNode_2009;
        }
        // simpan head ke variabel sementara
        NodeSLL_2511532009 last_2009 = head_2009;
        // telusuri ke node akhir
        while (last_2009.next_2009 != null) {
            last_2009 = last_2009.next_2009;
        }
        // ubah pointer
        last_2009.next_2009 = newNode_2009;
        return head_2009;
    }

    static NodeSLL_2511532009 GetNode_2009(int data_2009) {
        return new NodeSLL_2511532009(data_2009);
    }

    static NodeSLL_2511532009 insertPos_2009(NodeSLL_2511532009 headNode_2009, int
position_2009, int value_2009) {
        NodeSLL_2511532009 head_2009 = headNode_2009;
        if (position_2009 < 1)
            System.out.println("Invalid Position");
        if (position_2009 == 1) {
            NodeSLL_2511532009 new_node_2009 = new NodeSLL_2511532009(value_2009);
            new_node_2009.next_2009 = head_2009;
            return new_node_2009;
        } else {
            while (position_2009-- != 0) {
                if (position_2009 == 1) {
                    NodeSLL_2511532009 newNode_2009 = GetNode_2009(value_2009);
                    newNode_2009.next_2009 = headNode_2009.next_2009;
                    headNode_2009.next_2009 = newNode_2009;
                    break;
                }
                headNode_2009 = headNode_2009.next_2009;
            }
            if (position_2009 != 1)
                System.out.println("Posisi di luar jangkauan");
        }
    }
}
```



```

    }
    return head_2009;
}

public static void printList_2009(NodeSLL_2511532009 head_2009) {
    NodeSLL_2511532009 curr_2009 = head_2009;
    while (curr_2009.next_2009 != null) {
        System.out.print(curr_2009.data_2009 + "-->");
        curr_2009 = curr_2009.next_2009;
    }
    if (curr_2009.next_2009 == null) {
        System.out.print(curr_2009.data_2009);
    }
    System.out.println();
}

public static void main(String[] args) {
    // buat linked list 2->3->5->6
    NodeSLL_2511532009 head_2009 = new NodeSLL_2511532009(2);
    head_2009.next_2009 = new NodeSLL_2511532009(3);
    head_2009.next_2009.next_2009 = new NodeSLL_2511532009(5);
    head_2009.next_2009.next_2009.next_2009 = new NodeSLL_2511532009(6);

    // cetak list asli
    System.out.print("Senarai berantai awal: ");
    printList_2009(head_2009);

    //tambahkan node baru didepan
    System.out.print("tambah 1 simpul di depan: ");
    int data_2009 = 1;
    head_2009 = insertAtFront_2009(head_2009, data_2009);

    // cetak update list
    printList_2009(head_2009);

    System.out.print("tambah 1 simpul di belakang : ");
    int data2_2009 = 7;
    head_2009 = insertAtEnd_2009(head_2009, data2_2009);
    // cetak update list
    printList_2009(head_2009);

    System.out.print("tambah 1 simpul ke data 4: ");
    int data3_2009 = 4;
    int pos_2009 = 4;
    head_2009 = insertPos_2009(head_2009, pos_2009, data3_2009);
    printList_2009(head_2009);
}
}

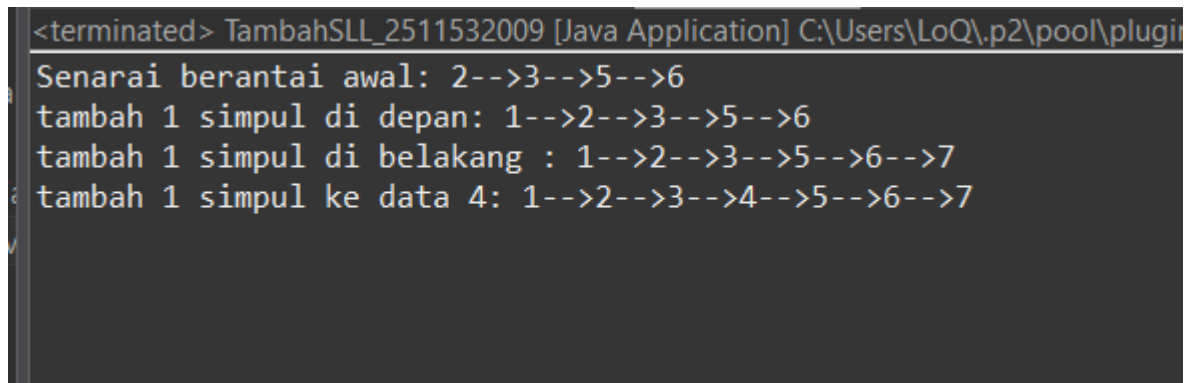
```

Program ini adalah contoh implementasi **linked list sederhana** dengan fokus pada operasi penambahan simpul di berbagai posisi. Pertama, terdapat method `insertAtFront_2009` yang menambahkan simpul baru di bagian depan list dengan cara menghubungkan node baru ke head lama. Kemudian ada `insertAtEnd_2009`, yang bertugas menambahkan simpul di bagian akhir. Jika list masih kosong, node

baru otomatis menjadi head. Jika tidak, program menelusuri list hingga simpul terakhir menggunakan perulangan while, lalu menyambungkan simpul baru di ujung.

Selain itu, method insertPos_2009 digunakan untuk menyisipkan simpul pada posisi tertentu. Jika posisi yang diminta adalah 1, simpul baru langsung dijadikan head. Jika posisinya lebih besar, program menelusuri list hingga mencapai simpul yang sesuai, lalu menyisipkan node baru di sana. Jika posisi tidak valid, program menampilkan pesan error. Untuk menampilkan isi list, method printList_2009 digunakan. Ia menelusuri setiap simpul dari head hingga akhir, mencetak nilai dengan tanda panah (-->) sebagai pemisah.

Di dalam main, program membuat list awal berisi 2->3->5->6. Setelah itu, simpul bernilai 1 ditambahkan di depan sehingga list menjadi 1->2->3->5->6. Kemudian simpul bernilai 7 ditambahkan di belakang, menghasilkan 1->2->3->5->6->7. Terakhir, simpul bernilai 4 disisipkan pada posisi ke-4, sehingga list akhir menjadi 1->2->3->4->5->6->7. Dengan demikian, program ini memperlihatkan bagaimana operasi penambahan simpul di depan, belakang, dan posisi tertentu dapat dilakukan dalam struktur data singly linked list. Dengan hasil output adalah sebagai berikut.



```
<terminated> TambahSLL_2511532009 [Java Application] C:\Users\LoQ\p2\pool\plugin
Senarai berantai awal: 2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang : 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

Gambar 1

2.2.3 Pencarian SLL

Untuk program dari PencarianSLL adalah sebagai berikut.

```
package pekan5_2511532009;

public class PencarianSLL_2511532009 {
    static boolean searchKey_2009(NodeSLL_2511532009 head_2009, int key_2009) {
        NodeSLL_2511532009 curr_2009 = head_2009;
        while (curr_2009 != null) {
            if (curr_2009.data_2009 == key_2009)
                return true;
            curr_2009 = curr_2009.next_2009;
        }
        return false;
    }

    public static void traversal_2009(NodeSLL_2511532009 head_2009) {
        // mulai dari head
        NodeSLL_2511532009 curr_2009 = head_2009;
        // telusuri sampai pointer null
        while (curr_2009 != null) {
            System.out.print(" " + curr_2009.data_2009);
            curr_2009 = curr_2009.next_2009;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        NodeSLL_2511532009 head_2009 = new NodeSLL_2511532009(14);
        head_2009.next_2009 = new NodeSLL_2511532009(21);
        head_2009.next_2009.next_2009 = new NodeSLL_2511532009(13);
        head_2009.next_2009.next_2009.next_2009 = new NodeSLL_2511532009(30);
        head_2009.next_2009.next_2009.next_2009.next_2009 = new
NodeSLL_2511532009(10);
        System.out.print("Penelusuran SLL :");
        traversal_2009(head_2009);
        // data yang akan dicari
        int key_2009 = 30;
        System.out.print("cari data " + key_2009 + " = ");
        if (searchKey_2009(head_2009, key_2009))
            System.out.println("Ketemu");
        else
            System.out.println("tidak ada");
    }
}
```

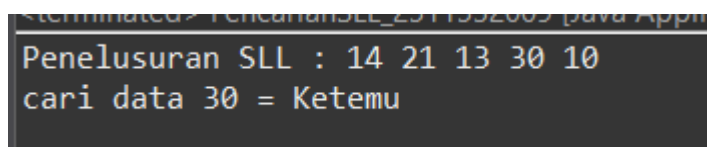
Program PencarianSLL_2511532009 ini dimulai dengan deklarasi method searchKey_2009 yang bertipe boolean. Method ini berfungsi untuk mencari data tertentu dalam linked list. Proses pencarian dilakukan dengan perulangan while selama simpul yang ditunjuk oleh variabel curr_2009 tidak bernilai kosong. Di

dalam perulangan, terdapat percabangan if yang membandingkan nilai data pada simpul saat ini dengan nilai kunci (key_2009). Jika sama, maka method langsung mengembalikan true sebagai tanda data ditemukan. Jika simpul terakhir sudah dilewati tanpa hasil, maka method akan mengembalikan false.

Selanjutnya, terdapat method traversal_2009 yang digunakan untuk menampilkan isi linked list. Method ini juga menggunakan perulangan while untuk menelusuri setiap simpul dari head hingga simpul terakhir, lalu mencetak nilai data yang tersimpan di setiap simpul dengan spasi sebagai pemisah.

Pada bagian main, program membentuk sebuah linked list berisi data 14 -> 21 -> 13 -> 30 -> 10. Setelah itu, program memanggil traversal_2009 untuk menampilkan isi list. Kemudian dilakukan pencarian terhadap data bernilai 30 dengan memanggil searchKey_2009. Karena data tersebut memang ada di dalam list, maka hasil pencarian menampilkan “Ketemu”. Jika data yang dicari tidak ada, maka program akan menampilkan pesan “tidak ada”.

Dengan demikian, program ini memperlihatkan cara kerja traversal untuk menelusuri isi list dan searching untuk menemukan data tertentu, serta memberikan output yang jelas apakah data berhasil ditemukan atau tidak. Dengan output yang dihasilkan adalah sebagai berikut.



```
<terminated> PenelusuranSLL_2511532009 [java Applic
Penelusuran SLL : 14 21 13 30 10
cari data 30 = Ketemu
```

Gambar 2

2.2.4 Hapus SLL

Untuk program dari HapusSLL adalah sebagai berikut.

```
package pekan5_2511532009;

public class HapusSLL_2511532009 {
    // fungsi untuk menghapus head
    public static NodeSLL_2511532009 deleteHead_2009(NodeSLL_2511532009 head_2009)
    {
        // jika SLL kosong
        if (head_2009 == null)
            return null;
        // pindahkan head ke node berikutnya
        head_2009 = head_2009.next_2009;
        //return head baru
    }
}
```

```

        return head_2009;
    }

    // fungsi menghapus node terakhir SLL
    public static NodeSLL_2511532009 removeLastNode_2009(NodeSLL_2511532009
head_2009) {
        // jika list kosong, return null
        if (head_2009 == null) {
            return null;
        }
        // jika list satu node, hapus node dan return null
        if (head_2009.next_2009 == null) {
            return null;
        }
        // temukan node terakhir terakhir ke dua
        NodeSLL_2511532009 secondLast_2009 = head_2009;
        while (secondLast_2009.next_2009.next_2009 != null) {
            secondLast_2009 = secondLast_2009.next_2009;
        }
        // hapus node terakhir
        secondLast_2009.next_2009 = null;
        return head_2009;
    }

    // fungsi menghapus node diposisi tertentu
    public static NodeSLL_2511532009 deleteNode_2009(NodeSLL_2511532009 head_2009,
int position_2009) {
        NodeSLL_2511532009 temp_2009 = head_2009;
        NodeSLL_2511532009 prev_2009 = null;
        // jika linked list null
        if (temp_2009 == null)
            return head_2009;
        // kasus 1 : head dihapus
        if (position_2009 == 1) {
            head_2009 = temp_2009.next_2009;
            return head_2009;
        }
        // kasus 2: menghapus node di tengah
        // telusuri ke node yang dihapus

        for (int i = 1; temp_2009 != null && i < position_2009; i++) {
            prev_2009 = temp_2009;
            temp_2009 = temp_2009.next_2009;
        }
        // jika ditemukan, hapus node
        if (temp_2009 != null) {
            prev_2009.next_2009 = temp_2009.next_2009;
        } else {
            System.out.println("Data tidak ada");
        }
        return head_2009;
    }

    // fungsi mencetak SLL
    public static void printList_2009(NodeSLL_2511532009 head_2009) {
        NodeSLL_2511532009 curr_2009 = head_2009;
        while (curr_2009.next_2009 != null) {
            System.out.print(curr_2009.data_2009 + "-->");
            curr_2009 = curr_2009.next_2009;
        }
    }

```

```

    }
    if (curr_2009.next_2009 == null) {
        System.out.print(curr_2009.data_2009);
    }
    System.out.println();
}
// kelas main
public static void main(String[] args) {
    // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> -> null
    NodeSLL_2511532009 head_2009 = new NodeSLL_2511532009(1);
    head_2009.next_2009 = new NodeSLL_2511532009(2);
    head_2009.next_2009.next_2009 = new NodeSLL_2511532009(3);
    head_2009.next_2009.next_2009.next_2009 = new NodeSLL_2511532009(4);
    head_2009.next_2009.next_2009.next_2009.next_2009 = new
NodeSLL_2511532009(5);
    head_2009.next_2009.next_2009.next_2009.next_2009.next_2009 = new
NodeSLL_2511532009(6);

    // cetak list awal
    System.out.println("List awal : ");
    printList_2009(head_2009);
    // hapus head
    head_2009 = deleteHead_2009(head_2009);
    System.out.println("List setelah head dihapus : ");
    printList_2009(head_2009);
    // hapus node terakhir
    head_2009 = removeLastNode_2009(head_2009);
    System.out.println("List setelah simpul terakhir dihapus : ");
    printList_2009(head_2009);
    // deleting node at position 2
    int position_2009 = 2;
    head_2009 = deleteNode_2009(head_2009, position_2009);
    // print list after deletion
    System.out.println("List setelah posisi 2 dihapus: ");
    printList_2009(head_2009);
}
}

```

Program ini diawali dengan inisialisasi **NodeSLL** sebagai struktur dasar untuk menyimpan simpul dalam linked list. Pada tahap awal terdapat percabangan if untuk memeriksa apakah head bernilai kosong (null). Jika head kosong, maka program akan mengembalikan nilai null sebagai tanda list tidak memiliki elemen.

Selanjutnya terdapat method `removeLastNode` yang berfungsi menghapus simpul terakhir dari list. Pertama, method ini mengecek apakah list kosong. Jika list hanya memiliki satu simpul, maka simpul tersebut dihapus dengan mengembalikan null. Namun, jika jumlah simpul lebih dari satu, program akan menelusuri list menggunakan variabel sementara `secondLast`.

Dengan perulangan while, simpul kedua terakhir ditemukan, lalu pointer simpul tersebut diubah menjadi null sehingga simpul terakhir terhapus.

Program juga memiliki method deleteNode yang digunakan untuk menghapus simpul dengan nilai tertentu. Selain itu, terdapat method printList yang bertugas menampilkan isi list dengan menelusuri setiap simpul dari head hingga akhir.

Pada bagian main, program mulai dengan menambahkan beberapa simpul ke dalam list. Setelah itu, isi list ditampilkan menggunakan printList. Kemudian dilakukan penghapusan head dengan memanggil method deleteHead. Untuk menghapus simpul terakhir, program memanggil removeLastNode. Terakhir, program juga memanggil deleteNode untuk menghapus simpul pada posisi ke-2. Setelah semua operasi penghapusan dilakukan, list ditampilkan kembali untuk menunjukkan hasil akhir setelah perubahan. Dengan output sebagai berikut

```
terminated: hapasLL_2311332003.java Application Crashed
List awal :
1-->2-->3-->4-->5-->6
List setelah head dihapus :
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus :
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

Gambar 3

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Single Linked List merupakan salah satu bentuk struktur data dinamis yang tersusun dari kumpulan node yang saling terhubung melalui pointer satu arah. Setiap node memiliki dua komponen utama, yaitu data yang disimpan dan referensi (*next*) yang menunjuk ke node berikutnya. Berbeda dengan array yang berukuran tetap dan dapat diakses langsung melalui indeks, linked list lebih fleksibel karena memori dialokasikan sesuai kebutuhan dan penelusuran hanya bisa dilakukan secara berurutan dari simpul awal (*head*) hingga simpul akhir (*null*). Karena sifatnya linear, manipulasi pointer menjadi hal yang sangat penting, sebab kesalahan dalam pengaturan referensi dapat menyebabkan rantai data terputus atau informasi hilang.

Keunggulan utama dari single linked list adalah kemampuannya dalam mengelola memori secara efisien, terutama saat melakukan operasi penyisipan maupun penghapusan di bagian ujung list tanpa harus menggeser elemen lain seperti pada array. Meski demikian, struktur ini memiliki keterbatasan, misalnya tidak mendukung akses acak (harus ditelusuri satu per satu) dan membutuhkan memori tambahan untuk menyimpan pointer. Walau ada kekurangan, single linked list tetap menjadi dasar penting dalam memahami konsep alokasi memori dinamis dan pengelolaan referensi. Penguasaan logika pointer serta penanganan kondisi list kosong tidak hanya melatih ketelitian dalam pemrograman, tetapi juga menjadi bekal untuk mempelajari struktur data yang lebih kompleks di tahap berikutnya.

3.2 Saran

- Ulangi dan terus Latihan agar makin memahami konsep dari Linked List.
- Sertakan Langkah kerja program secara ruhut dan sistematis.

BAB IV LAPORAN TUGAS

4.1 Kode Program

```
package pekan5_2511532009; // Package untuk mengelompokkan kelas sesuai NIM

// Kelas ADT Pasien yang merepresentasikan node dalam Single Linked List
public class Pasien_2511532009 {
    // Atribut data pasien dan pointer next dengan suffix NIM
    private String nama_2009;
    private String penyakit_2009;
    private int no_2009;
    private Pasien_2511532009 next_2009;

    // Constructor untuk menginisialisasi data pasien saat node dibuat
    public Pasien_2511532009(String nama_2009, String penyakit_2009, int no_2009)
    {
        this.nama_2009 = nama_2009;
        this.penakit_2009 = penyakit_2009;
        this.no_2009 = no_2009;
        this.next_2009 = null;
    }

    // Getter: mengambil nilai atribut
    public String getNama_2009() { return nama_2009; }
    public String getPenyakit_2009() { return penyakit_2009; }
    public int getNomor_2009() { return no_2009; }
    public Pasien_2511532009 getNext() { return next_2009; }

    // Setter: mengubah nilai atribut atau menghubungkan pointer next
    public void setNama_2009(String nama_2009) { this.nama_2009 = nama_2009; }
    public void setPenyakit_2009(String penyakit_2009) { this.penakit_2009 =
penyakit_2009; }
    public void setNomor_2009(int no_2009) { this.no_2009 = no_2009; }
    public void setNext_2009(Pasien_2511532009 next_2009) { this.next_2009 =
next_2009; }
}
```

4.2 Driver Program

```
package pekan5_2511532009;

import java.util.Scanner;

public class RumahSakit_2511532009 {

    static int counterAntrian_2009 = 0;

    public static Pasien_2511532009 insertAtTail_2009(Pasien_2511532009 head_2009,
String nama_2009, String penyakit_2009) {
        counterAntrian_2009++;
    }
}
```

```

        Pasien_2511532009 newNode_2009 = new Pasien_2511532009(nama_2009,
penyakit_2009, counterAntrian_2009);

        if (head_2009 == null) {
            return newNode_2009;
        }

        Pasien_2511532009 last_2009 = head_2009;
        while (last_2009.getNext() != null) {
            last_2009 = last_2009.getNext();
        }

        last_2009.setNext_2009(newNode_2009);
        return head_2009;
    }

    // Delete Head: menghapus pasien terdepan dan menggeser head ke simpul
berikutnya
    public static Pasien_2511532009 deleteHead_2009(Pasien_2511532009 head_2009) {
        if (head_2009 == null) {
            System.out.println("Antrian kosong! Tidak ada pasien yang bisa
dipanggil.");
            return null;
        }

        Pasien_2511532009 pasienDipanggil_2009 = head_2009;
        System.out.println("\n=== Pasien Dipanggil ===");
        System.out.println("Nama          : " +
pasienDipanggil_2009.getNama_2009());
        System.out.println("Penyakit       : " +
pasienDipanggil_2009.getPenyakit_2009());
        System.out.println("Nomor Antrian : " +
pasienDipanggil_2009.getNomor_2009());

        return head_2009.getNext();
    }

    // Display List: menampilkan seluruh data pasien dalam antrian
    public static void printList_2009(Pasien_2511532009 head_2009) {
        if (head_2009 == null) {
            System.out.println("Antrian kosong. Belum ada pasien terdaftar.");
            return;
        }

        System.out.println("\n=== Daftar Antrian ===");
        Pasien_2511532009 curr_2009 = head_2009;

        while (curr_2009 != null) {
            System.out.println("[ " + curr_2009.getNomor_2009() + " ] " +
curr_2009.getNama_2009() + " - " +
curr_2009.getPenyakit_2009() +
(curr_2009.getNext() != null ? " -->" : ""));
            curr_2009 = curr_2009.getNext();
        }
    }

    // Search Case-Insensitive: mencari pasien berdasarkan nama
    public static void searchPatient_2009(Pasien_2511532009 head_2009, String
namaCari_2009) {

```

```

        if (head_2009 == null) {
            System.out.println("Antrian kosong.");
            return;
        }

        Pasien_2511532009 curr_2009 = head_2009;
        boolean ditemukan_2009 = false;

        while (curr_2009 != null) {
            if (curr_2009.getNama_2009().equalsIgnoreCase(namaCari_2009)) {
                System.out.println("\n=== Pasien Ditemukan ===");
                System.out.println("Nama          : " + curr_2009.getNama_2009());
                System.out.println("Penyakit       : " +
curr_2009.getPenyakit_2009());
                System.out.println("Nomor Antrian : " +
curr_2009.getNomor_2009());
                ditemukan_2009 = true;
                break;
            }
            curr_2009 = curr_2009.getNext();
        }

        if (!ditemukan_2009) {
            System.out.println("Pasien dengan nama '" + namaCari_2009 + "' tidak
ditemukan.");
        }
    }

    // Cek Status Antrian: menghitung total pasien dan menampilkan pasien terdepan
    public static void checkStatus_2009(Pasien_2511532009 head_2009) {
        if (head_2009 == null) {
            System.out.println("Status Antrian: KOSONG. Tidak ada pasien.");
            return;
        }

        int total_2009 = 0;
        Pasien_2511532009 curr_2009 = head_2009;
        while (curr_2009 != null) {
            total_2009++;
            curr_2009 = curr_2009.getNext();
        }

        System.out.println("\n=== Status Antrian ===");
        System.out.println("Total Pasien      : " + total_2009);
        System.out.println("Pasien Terdepan : " + head_2009.getNama_2009() +
            " (No. " + head_2009.getNomor_2009() + ")");
    }

    // Menu utama
    public static void main(String[] args) {
        Scanner input_2009 = new Scanner(System.in);
        Pasien_2511532009 head_2009 = null;
        int pilihan_2009 = 0;

        do {
            System.out.println("\n=== Antrian Rumah Sakit NIM: 2511532009 ===");
            System.out.println("1. Daftarkan Pasien (Insert at Tail)");
            System.out.println("2. Panggil Pasien (Delete Head)");
            System.out.println("3. Tampilkan Antrian (Display)");

```

```

        System.out.println("4. Cari Pasien (Search)");
        System.out.println("5. Cek Status Antrian");
        System.out.println("6. Keluar");
        System.out.print("Pilihan: ");

        if (input_2009.hasNextInt()) {
            pilihan_2009 = input_2009.nextInt();
            input_2009.nextLine();
        } else {
            System.out.println("Input tidak valid. Silakan masukkan angka 1-6.");
            input_2009.nextLine();
            continue;
        }

        switch (pilihan_2009) {
            case 1:
                System.out.print("Masukkan Nama Pasien: ");
                String nama_2009 = input_2009.nextLine();
                System.out.print("Masukkan Penyakit : ");
                String penyakit_2009 = input_2009.nextLine();
                head_2009 = insertAtTail_2009(head_2009, nama_2009,
                penyakit_2009);
                System.out.println("Pasien berhasil didaftarkan! Nomor
                Antrian: " + counterAntrian_2009);
                break;
            case 2:
                head_2009 = deleteHead_2009(head_2009);
                break;
            case 3:
                printList_2009(head_2009);
                break;
            case 4:
                System.out.print("Masukkan Nama Pasien yang dicari: ");
                String cari_2009 = input_2009.nextLine();
                searchPatient_2009(head_2009, cari_2009);
                break;
            case 5:
                checkStatus_2009(head_2009);
                break;
            case 6:
                System.out.println("Program selesai. Terima kasih.");
                break;
            default:
                System.out.println("Pilihan tidak valid. Silakan pilih 1-6.");
        }
    } while (pilihan_2009 != 6);

    input_2009.close();
}
}

```

4.3 Output yang dihasilkan

```
=== Antrian Rumah Sakit NIM: 2511532009 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Wahyu
Masukkan Penyakit   : Demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511532009 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: Fajri
Masukkan Penyakit   : Suka Beribadah
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511532009 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama Pasien yang dicari: Wahyu
```

Gambar 4

```
=== Antrian Rumah Sakit NIM: 2511532009 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5

=== Status Antrian ===
Total Pasien      : 2
Pasien Terdepan   : Wahyu (No. 1)

=== Antrian Rumah Sakit NIM: 2511532009 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2

=== Pasien Dipanggil ===
Nama              : Wahyu
Penyakit          : Demam
Nomor Antrian     : 1

=== Antrian Rumah Sakit NIM: 2511532009 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Program selesai. Terima kasih.
```

Gambar 5

DAFTAR PUSTAKA

- [1] Modul Praktikum Struktur Data Pekan 5
- [2] Data Structures and Algorithms in Java